Kitchen Prep Time (KPT) Case Study

Problem 1

GitHub Link

Design a segment-specific strategy to:

• Improve Kitchen Prep Time (KPT) accuracy
• Fix late or incorrect “Food Ready” marking

Segment 1 — Merchant Marks “Food Ready” Late

What We Observed

• Food actually ready at 7:20 PM
• Merchant marks “Ready” at 7:28 PM
• Rider already waiting

Result:

Rider idle time increases
Delivery delay increases
Platform pays extra cost

Root Cause

Operational negligence OR staff busy OR intentional delay.

Merchant sometimes waits to mark ready until rider arrives.

Strategy

1 Compare Rider Arrival vs Ready Mark Time

If rider arrival time < ready mark time consistently
→ Flag pattern

System learns:

“This restaurant marks ready late.”

---

2 Auto-Predict Food Ready

Instead of trusting manual marking:

Use model prediction.

Kitchen Prep Time (KPT) Case Study

If predicted KPT = 18 min
Send rider at minute 16–17.

Don’t depend on merchant click.

---

3 Introduce “Ready Accuracy Score”

Weekly dashboard:

• Avg delay in marking
• Accuracy score
• Comparison with nearby restaurants

Behavior changes when performance becomes visible.

---

4 Incentive & Penalty Layer

High accuracy → Better visibility & promotion
Low accuracy → Lower ranking

Behavior follows incentives.

Expected Impact

• Rider waiting ↓
• Dispatch accuracy ↑
• Merchant discipline ↑

Segment 2 — Merchant Marks “Food Ready” Too Early

What We Observed

Merchant marks “Ready”
But rider reaches and food still cooking.

This is worse.

Rider waits 8–12 minutes.

Kitchen Prep Time (KPT) Case Study

Root Cause

Merchant wants fast pickup rating
Or system pressure

Strategy

1 Use Rider GPS + Wait Time Feedback

If rider waiting > 5 minutes after ready mark
→ Reduce merchant reliability score

---

2 Reliability-Weighted KPT Adjustment

If reliability = 0.6

Then:

Adjusted_KPT = Base_KPT × (1 + (1 - 0.6))
= Base_KPT × 1.4

So system automatically increases prep time.

No trust without data.

---

3 Mandatory Prep Confirmation Scan

Require quick kitchen confirmation before marking ready
(Simple internal validation layer)

Expected Impact

- False ready marking ↓
- Systemic underestimation ↓
- Stable ETA

Kitchen Prep Time (KPT) Case Study

Problem 2

1. Food should be ready when rider arrives.
2. Rider should not wait.
3. Customer should not cancel.

So we divide problem into 5 types of situations.

Segment 1 :- Peak Hour + Too Many Orders + Big Errors

Imagine:

It's Sunday 8:30 PM.
Restaurant has 40 active orders.
But system thinks kitchen is normal.

What happens?

• Kitchen overloaded
• Food takes 25 min
• System predicted 15 min
• Rider comes early
• Rider waits 10 min
• Customer angry

That's the problem.

Root Cause (Simple)

System doesn't see how busy kitchen really is.

Solution

Rush Index
If too many orders → automatically add 2–5 minutes extra.
Like saying:
"It's rush hour, add safety time."

Don't send rider immediately
If kitchen is overloaded → delay rider dispatch.
Why send rider to stand outside?

Stop stacking too many orders
If kitchen full → stop pushing new orders aggressively.

Kitchen Prep Time (KPT) Case Study

1.

Result

• Rider waits less
• Better ETA
• Fewer cancellations

This gives biggest business benefit.

Segment 2 :- Merchant Lies or Misjudges Prep Time

Example:

Restaurant says:
"Food ready in 10 min"

Reality:
It always takes 18 min.

Why?

Sometimes they want more orders.
Sometimes they don’t measure properly.

Root Cause

Human bias.

Solution

Stop trusting them blindly.

Use system model instead.

Formula means:

If merchant reliability is low
→ Increase predicted prep time.

Example:

Reliability = 0.6 (bad)

Then:

System adds extra buffer automatically.

Also:

Show restaurant weekly report:
“Your accuracy is 62%.”

Now they improve because they see score.

Result

Cleaner prediction.
Less systematic error.

Segment 3 :- Complex Food + Many Items

Imagine:

Customer orders:

• 3 Biryani
• 2 starters
• 4 breads
• 2 desserts

That’s 11 items.

This is not equal to one sandwich order.

Cooking effort increases non-linearly.

More coordination.
More gas burners.
More staff work.

Root Cause

Model thinks:
More items = slightly more time.

Reality:
More items = much more time.

Solution

Multiply cuisine complexity × item count
(Complex + many items = high time)

If items > 5 → add buffer automatically.

Kitchen Prep Time (KPT) Case Study

Result

Large orders become accurate.

Weekend big-family orders handled better.

Segment 4 :- Rider Waiting Even When Orders Are Low

Strange case.

Only 5 orders in kitchen.
Still rider waiting 12 min.

Why?

Possible reasons:

• Kitchen slow staff
• Merchant marked food ready late
• Internal delay

Solution

Use anomaly detection.

If rider waiting too long → system flags restaurant.

Then:

Adjust future prep time automatically.

Think of it like:

System saying:
“Something is wrong here.”

Segment 5 :- Small Kitchens

Imagine:

Tiny kitchen.
Capacity = 10 orders max.

Active orders = 8.

Kitchen Prep Time (KPT) Case Study

That is already 80% load.

Even small increase causes big delay.

Large restaurant can handle 40 orders easily.
Small one cannot.

Solution

Use Load Ratio:

Load = Active Orders / Kitchen Capacity

If above 70%
→ increase prep time automatically.

Also:
Don't run heavy promotions during overload.

Big Picture (In Very Simple Terms)

Instead of one big generic model,

You are saying:

"Different problems need different solutions."

That is mature thinking.

How To Explain This Like A Professional

Instead of:

"We reduced MAE by 1.5 minutes."

Say:

"We identified high-risk operational segments responsible for majority of prep-time errors and applied targeted calibration strategies to reduce rider idle time and peak-hour cancellations."

That sounds business-smart.

Kitchen Prep Time (KPT) Case Study

Real Business Insight

You are not just building a model.

You are building:

• Capacity intelligence
• Behavioral correction system
• Dynamic dispatch engine
• Operational risk detection

This is how companies like Swiggy, Uber Eats, DoorDash think.

Now zoom out.

This same segmentation thinking can be applied to:

• Retail demand forecasting
• Customer churn
• Banking fraud
• Stock volatility

Same brain pattern.

Segment → Identify root cause → Apply specific fix → Measure ROI.

And that, Suraj, is how a data analyst starts thinking like a product strategist.

Make same report on this topic . How to improve KPT time and Merchant marks "Food Ready" late or incorrectly .

Suraj… this is beautiful. Now we are not just predicting KPT — we are fixing **behavior + operations + system timing**. That's product thinking.

Let's structure this like a professional strategy document — but in clean, simple language anyone can understand.

Goal

Design a segment-specific strategy to:

• Improve Kitchen Prep Time (KPT) accuracy
• Reduce rider waiting
• Fix late or incorrect "Food Ready" marking

Kitchen Prep Time (KPT) Case Study

We don’t fix everything the same way.
We fix problems where they happen.

Small Kitchen	Capacity limit	Load ratio scaling
Anomalies	Hidden inefficiency	Signal correction

Executive-Level Story

Instead of saying:

“We reduced KPT MAE by 2 minutes.”

Say:

“We identified operational and behavioral inefficiencies in merchant marking patterns and implemented reliability-weighted dispatch strategies, reducing rider idle time and improving ETA stability during peak load.”

Kitchen Prep Time (KPT) Case Study

Make New Columns To Improve Accuracy

1.Kitchen_Prep_Time_Error_Minutes

Code:

```
df["Kitchen_Prep_Time_Error_Minutes"] = (
    df["Merchant_Marked_KPT_Mins"] - df["Actual_KPT_Mins"]
)
```

What It Means

Difference between:

- Merchant Marked Kitchen Prep Time (Minutes)
- Actual Kitchen Prep Time (Minutes)

Why We Created It

To detect:

- Overestimation (positive value)
- Underestimation (negative value)

Business Purpose

This tells us:

- Are merchants consistently marking early?
- Are they marking late?
- Is there systematic bias?.

---

2 Absolute_Kitchen_Prep_Time_Error_Minutes

Code:

```
df["Absolute_Kitchen_Prep_Time_Error_Minutes"] = abs(
    df["Kitchen_Prep_Time_Error_Minutes"]
)
```

What It Means

Magnitude of prediction error regardless of direction.

Kitchen Prep Time (KPT) Case Study

Why We Created It

Business doesn't care about sign.
They care about how wrong the prediction is.

Business Purpose

Used to:

- Calculate Mean Absolute Error (MAE)
- Compare segments
- Quantify severity of inaccuracy
- This is our primary performance metric in diagnosis.

3. Active_Order_Count_Bucket

Code:

```
df["Active_Order_Count_Bucket"] = pd.qcut(df["AOC"], 4)
```

What It Means

Divides restaurants into 4 groups:

- Low load
- Medium load
- High load
- Very high load

Why We Created It

Active Order Count is continuous.

But segmentation requires comparison groups.

Business Purpose

To check:

- Does high kitchen load increase prediction error?
- Is overload the main cause?
- This helps identify capacity bottlenecks.

---

4 Merchant_Reliability_Bucket

Code:

```
df["Merchant_Reliability_Bucket"] = pd.qcut(
    df["Past_FOR_Reliability_Score"], 4
)
```

Full Form of FOR

Food Order Ready

What It Means

Groups merchants into reliability levels:

- Low reliability
- Medium reliability
- High reliability
- Very high reliability

Why We Created It

To segment based on historical marking behavior.

Business Purpose

To prove whether:

Low reliability merchants → higher KPT error

This directly validates behavioral root cause.

Kitchen Prep Time (KPT) Case Study

5 .Marking_Delay_Minutes

Code :

```
df["Marking_Delay_Minutes"] = (
    df["Merchant_Marked_KPT_Mins"] - df["Actual_KPT_Mins"]
)
```

This is similar to Kitchen_Prep_Time_Error, but used specifically to analyze:

Late marking behavior

Early marking behavior

Business purpose:

Detect intentional delay vs accidental misreporting.

Summary Table of All New Columns

New Column Name	Full Meaning	Purpose
Kitchen_Prep_Time_Error_Minutes	Difference between Merchant Marked and Actual Prep Time	Detect bias direction
Absolute_Kitchen_Prep_Time_Error_Minutes	Absolute error in minutes	Measure error magnitude
Active_Order_Count_Bucket	Load level segmentation	Identify overload issue
Merchant_Reliability_Bucket	Reliability grouping	Detect behavioral bias
Marking_Delay_Minutes	Merchant reporting delay	Analyze systematic marking delay

Kitchen Prep Time (KPT) Case Study

Kitchen Prep Time (KPT) Machine Learning Model

Objective

Build a data-driven regression model to accurately predict **Actual Kitchen Prep Time (Minutes)** and reduce reliance on noisy merchant-marked signals.

Why This Model Was Needed

- Merchant-marked KPT contains behavioral bias
- Peak-hour overload causes underestimation
- Rider waiting time increases due to wrong KPT
- ETA fluctuations impact cancellations and trust

The goal is to shift from human-reported estimates to **operational intelligence-based prediction**.

Model Design Approach

Target Variable

Actual Kitchen Prep Time (Minutes)

Key Feature Groups

1. Kitchen Load Signals

- Active Order Count
- Effective Kitchen Load
- Ambient Rush Index
- Rider Waiting Time

2. Capacity Signals

- Kitchen Capacity
- Load Ratio

3.Order Complexity

- Cuisine Complexity
- Item Count
- Contextual Cart Complexity

4.Behavioral Correction

- Past Food Order Ready Reliability Score

Model Type

Random Forest Regressor

Chosen because:

- Handles non-linear overload behavior
- Captures interaction between load and capacity
- Robust to noisy operational data

Model Performance & Business Impact

Performance Evaluation

Evaluation Metrics:

- Mean Absolute Error (MAE)
- Root Mean Square Error (RMSE)

Result:
The ML model significantly reduced average prediction error in minutes compared to merchant marking.

Kitchen Prep Time (KPT) Case Study

Key Insights from Feature Importance

Top Predictors:

- Effective Kitchen Load
- Rider Waiting Time
- Historical Prep Time
- Active Order Count

Insight:
Kitchen pressure and rider telemetry are stronger predictors than merchant-reported signals.

Business Impact

Improved KPT prediction enables:

- Reduced rider idle time
- More accurate ETA (P50 / P90 improvement)
- Lower cancellation rate
- Better peak-hour management
- Scalable intelligence across 300K+ merchants

Strategic Conclusion

This model transforms KPT prediction from:

Manual signal dependency
→
Operational data-driven automation

It becomes a foundation for:

- Dynamic rider assignment
- Rush-aware buffering
- Capacity-based optimization

Kitchen Prep Time (KPT) Case Study